When Type Checking Goes Wrong, but Keeps Going

From Merlin to OCaml's Typechecker: Integrating Typing Recovery

Xavier Van de Woestyne
Tarides
Paris, France
xavier@tarides.com

Abstract

An interactive editing environment for a programming language closely resembles the front-end of its compiler. Merlin, the recommended *Language Server for OCaml*[1], therefore embeds a modified copy of the OCaml front-end. Unlike a compiler, which analyzes complete and syntactically valid programs, an editor continuously operates on incomplete programs that are temporarily inconsistent or ill-typed. To bridge this gap, Merlin relies on two recovery mechanisms. **Parsing Recovery** synthesizes AST nodes that allow parsing to continue after syntax errors, while **Typing Recovery** synthesizes typed AST nodes that allow type checking to continue after type errors. These mechanisms are currently implemented as a collection of patches maintained on top of each supported version of the OCaml type-checker.

This presentation describes the work that led to the integration of the **Typing Recovery** mechanism directly into the OCaml type-checker, in order to reduce divergence between Merlin and the compiler, simplify code maintenance, and align their evolution cycles.

1 Introduction

OCaml has a powerful type system based on type inference. The role of the type-checker is to determine whether a program is well-typed in a given environment. When a type error is encountered, the type derivation can no longer be continued and the compiler stops by reporting the first observed error. This strategy is well suited to compilation, whose goal is to decide whether a program is valid. The situation is different in an interactive editing environment. The program manipulated by the user is rarely complete[2] and frequently contains temporary syntax or type errors. Yet the language server must continue to provide useful information[3], such as type display, autocompletion, or the localization of multiple simultaneous errors. It therefore no longer seeks only to answer the question *"is this module correctly typed in the current environment?"*, but rather the following question: *"could this module become correctly typed after a few fixes?"*

To address this issue, Merlin adapts the compiler front-end using two *recovery* mechanisms that allow multiple errors to be reported (rather than stopping at the first one) and that keep editor features available on a partially invalid document.

- **Parsing Recovery**, which synthesizes syntactic nodes in order to continue analysis despite syntax errors, always producing an AST (`Parsetree`) for any input.
- **Typing Recovery**, which synthesizes artificial typed nodes in order to continue analysis despite type errors, always producing a typed AST (`Typedtree`) for any `Parsetree`.

These two mechanisms are essential to Merlin's correct functioning and to the quality of the user experience. However, their maintenance is complex. Each new version of the OCaml compiler requires reapplying recovery patches on both the parser and the type-checker, which introduces a gradual risk of divergence between Merlin and the official compiler. Moreover, some language evolutions may require a deep redesign of the recovery mechanisms. Until now, Merlin maintained a modified copy of OCaml's parser and type-checker. Each compiler evolution had to be propagated into this copy, introducing a significant duplication of the front-end and a non-negligible maintenance cost.

In order to reduce the difference between the OCaml front-end and Merlin's modified front-end, the **Typing Recovery** mechanism has been integrated directly into the OCaml compiler type-checker [4]. This integration removes code duplication between the two systems and reduces the maintenance cost associated with compiler evolution. It also comes with a clarification of the existing mechanism and a redesign of its structure, in order to facilitate future evolution and to make recovery-related design choices explicit and systematic within the core of the type-checker.

2 How Typing Recovery Works

The core of the OCaml type-checker is implemented as a set of recursive functions responsible for typing the different syntactic categories of the language, notably expressions, patterns, type declarations, modules, and classes. When a type error is encountered, these functions propagate an exception that interrupts the construction of the typing derivation. The compiler thus adopts a *first-failure reporting* strategy, consisting in reporting only the first encountered error. This approach is particularly well suited to OCaml, whose type system based on inference makes it difficult to continue analysis without risking the propagation of imprecise diagnostics.

This strategy, well adapted to compilation, is however less satisfactory in the context of a language server, where the analyzed program is, by nature, incomplete and temporarily incorrect.

2.1 Generic Recovery

To go beyond the *first-failure reporting* strategy, Merlin instruments the main recursive functions of the type-checker in order to intercept typing exceptions. When an exception is raised, it is caught so

that it can be recorded, and typing then resumes at the recursive call level (locally) on the other independent subterms of the program. Thus, even if the typing of a subexpression fails, independent subtrees can still be analyzed, which makes it possible to report multiple typing errors within a single analysis.

Resuming recursion allows new errors to be discovered, but is not sufficient to produce an exploitable `Typedtree`. When a subderivation fails, Merlin synthesizes an artificial `Typedtree` node whose typing judgment is that expected by the context in which it is inserted, while also preserving the fragments of the derivation that could be constructed before the typing failure, even if their attachment to the rest of the tree is no longer known. These nodes help preserve the structure of the `Typedtree` returned by the type-checker and maintain language server functionality despite the presence of errors.

2.2 Fine-grained recovery

However, these generic mechanisms are not sufficient to cover all language constructs. Some phases of the type-checker perform preliminary checks before recursively typing their subterms. When these checks fail, recursion is never engaged and no partial derivation is produced. For example, record expressions:

```
{ my_fst_field = e1;
  my_snd_field = e2 }
```

The type-checker first attempts to resolve `my_fst_field` and `my_snd_field`. If, for instance, `my_fst_field` does not exist, it gives up without attempting to type `e1` and `e2`. This strategy is suboptimal: subexpressions are never analyzed even though they could contain useful information or independent errors. To address this issue, Merlin delays the error by assuming that the field exists.

Merlin then temporarily adapts the type-checker by considering that an unknown label is not necessarily a fatal error, but may correspond to a field that will be added later. Typing then proceeds under this assumption, which makes it possible to produce partial derivations for subexpressions and to improve the quality of diagnostics.

2.3 “Limitations” of the Current Approach

This approach has allowed Merlin to provide, for many years, a high-quality user experience. However, it has a few limitations that complicate its evolution and maintenance:

- The Typing Recovery mechanism relies on a modified copy of the OCaml type-checker maintained within Merlin. Each compiler evolution therefore requires porting recovery patches onto this new version (installation of local exception handlers), which represents a significant maintenance cost and encourages gradual divergence between the two implementations.
- Adding new features to the type system often raises questions about their behavior in the presence of errors. These decisions are currently made within Merlin, after the feature has been developed in the compiler. However, the authors of the feature already have a deep understanding of its semantics and are often best placed to define a coherent recovery strategy.
- The Typing Recovery mechanism has progressively grown alongside language evolution. Some design decisions, made several years ago, are no longer documented and rely on implicit invariants. This situation complicates the evolution of the mechanism and makes the introduction of new features more difficult.

These limitations do not call into question the effectiveness of the Typing Recovery mechanism. However, they motivate a tighter integration within the compiler, in order to reduce front-end duplication, bring recovery decisions closer to the implementation of new features, and make the invariants of the mechanism more explicit.

3 Integration into the OCaml type checker

The integration of the *Typing Recovery* mechanism directly into the OCaml type-checker addresses the limitations presented in the previous section. First, it reduces the maintenance cost by removing most of the patches that Merlin had to apply for each new compiler version. It also brings recovery-related decisions closer to the implementation of language features, allowing their authors to define an error-handling strategy that is consistent with their design.

Finally, this integration provided an opportunity to revisit the historical *Typing Recovery* mechanism. Freed from the constraint of minimizing differences between Merlin and the compiler, the new implementation could be redesigned to make its invariants more explicit and to propose a more systematic architecture, thereby facilitating future evolution.

However, integrating the mechanism into the compiler does not simply consist in moving existing code. In order to preserve the properties expected by Merlin while limiting the impact on the compiler, several design goals guided the new implementation.

- **Optional.** The mechanism must be activatable only on demand, via the `-typing-recovery` flag. Since the diagnostics produced during recovery may be less precise than those of the compiler, this mode is intended for language servers and not for traditional compilation.
- **Systematic.** The API must make the presence of recovery explicit so that every evolution of the type-checker naturally takes this mechanism into account, without requiring ad hoc patches afterwards. it should make recovery an integral part of the type-checker, so that the introduction of a new typing rule naturally leads to defining its behavior in the presence of errors.
- **Simple.** The proposed API must remain easy to understand and use in order to limit the cognitive burden imposed on compiler maintainers.

3.1 Distinction between Generic and Fine-grained recovery

In order to make error handling explicit, we introduce two functions intended to replace direct calls to `raise` in the type-checker. When the `-typing-recovery` flag is not enabled, these functions behave exactly like `raise`. In *Typing Recovery* mode, they record the exception in a dedicated structure, which will then be processed by the recovery handler.

- `Typing_recovery.log_and_raise` records the exception before re-raising it. It is used when processing cannot continue locally and captures the general case.
- `Typing_recovery.log_or_raise` records the exception without re-raising it when the context is able to perform local recovery, for example by synthesizing an artificial node or by continuing the typing of other subterms; it captures more fine-grained cases.

This distinction makes explicit the points where recovery is possible and those where the exception must necessarily be propagated. The API thus guides the implementation of the type-checker and makes recovery decisions visible when adding or modifying typing rules[1].

3.2 When to recover

In Merlin's historical implementation, the selection of exceptions eligible to trigger a *Typing Recovery* scenario was mainly based on an empirical approach. The reimplementation in the compiler made it possible to identify a simple design invariant: only exceptions intended to be reported to the user (*user-facing errors*) should be recorded by the recovery mechanism. Conversely, internal type-checker exceptions, as well as simply re-raised exceptions (*reraise*), must not be captured.

3.3 Force logging of user-facing errors

To introduce recovery as a systematic approach, we aim to channel *user-facing* exceptions so that they are always handled through a pair of functions `log_and_raise` and `log_or_raise`. To achieve this, we rely on a property of the OCaml language: exceptions are represented by the extensible type `exn` [5]. This representation makes it possible to control the introduction of new exception constructors via module interfaces. By making certain exception constructors *private* in a module signature, it becomes possible to forbid their construction outside this module, while still allowing them to be deconstructed through pattern matching or exception handlers. This technique thus makes it possible to structure the flow of *user-facing* errors and to enforce a systematic passage through the associated logging functions.

The modules involved in type-checking expose a set of errors described by the `error` type and an `Error of error` exception allowing errors to interrupt the program's execution flow. The exception is removed and made private inside an internal module (named `Error`), making it impossible to use `raise (Error.In_context x)` outside the `Error` module, and thus enforcing the use of the two functions `log_and_raise` and `log_or_raise`.

```
- exception Error of error

+ module Error : sig
+    type exn += private In_context of error
+    val log_or_raise :  error -> unit
+    val log_and_raise : error -> 'a
+ end = struct
+   type exn += In_context of error
+
+   let log_or_raise err =
+     Typing_recovery.log_or_raise (In_context err)
+
+   let log_and_raise err =
+     Typing_recovery.log_and_raise (In_context err)
+ end
```

This transformation also makes it straightforward to rely on the compiler to guide the implementation by identifying and handling every possible recovery case individually.

3.4 Additional Work

We have outlined the overall design of Typing Recovery and its implementation in the OCaml type-checker. However, for the sake of conciseness, we have left several topics aside:

- The preservation of type variables generated by recovery: since recovery may generate type variables to synthesize `Typedtree` nodes, these variables are preserved in order to avoid cascades of errors induced by their introduction.
- The implementation details related to error logging and monitoring.
- The stabilization of types for display: because Typing Recovery does not interrupt type-checking, a type appearing in a logged error may subsequently evolve (for example, by being unified more precisely), thereby reducing the quality of error messages. The recovery mechanism therefore introduces a type-freezing mechanism at logging time, ensuring that types are not modified after the error message has been generated.

In addition, recovery was exercised on every error that may be propagated by the `Typecore` module, thereby covering a broad range of use cases and increasing confidence in its behavior.

4 Acknowledgments

The Typing Recovery mechanism was originally designed by Frédéric Bour and Thomas Refis, who maintained and evolved it over several years. More recently, its maintenance was taken over by Ulysse Gérard, who ensured the continuity of development on this component. Thomas Refis contributed to the present implementation by documenting the original mechanism, proposing design improvements, and providing several detailed code reviews. Florian Angeletti contributed through reviews and fixes, notably regarding the handling of deep type copying, making it possible to integrate this modification into the OCaml type-checker.

[1]Today, we do not have a formal criterion to systematically determine which function to use. The integration into the OCaml type-checker was therefore guided by a large set of unit tests and empirical validation. In practice, the choice between the two functions depends on the expected behavior of recovery and is validated by observing the produced diagnostics. This lack of a general criterion aligns with the observations of Bour *et al.*[1], who note that recovery policies would benefit from a more systematic theoretical framework.

5 Conclusion

We have presented the Typing Recovery mechanism and the transformations made to its original implementation, which originated in Merlin. This work has clarified its operation and made its integration into the addition of new typing features within OCaml's type system more systematic. The patch introducing the Typing Recovery mechanism has been merged into the compiler's `trunk` and will be available in OCaml version `5.6`.

The next step is to update Merlin so that it relies on this new recovery mechanism, thereby significantly reducing the divergence between the OCaml front-end used by Merlin and that of the official compiler.

References

[1] Frédéric Bour, Thomas Refis, and Gabriel Scherer. Merlin: A language server for ocaml (experience report). 2018.

[2] Cyrus Omar, Ian Voysey, Michael Hilton, Joshua Sunshine, Claire Le Goues, Jonathan Aldrich, and Matthew A. Hammer. Toward Semantic Foundations for Program Editors. In Benjamin S. Lerner, Rastislav Bodík, and Shriram Krishnamurthi, editors, *2nd Summit on Advances in Programming Languages (SNAPL 2017)*, volume 71 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 11:1–11:12, Dagstuhl, Germany, 2017. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[3] Xavier Van De Woestyne, Sonja Heinze, Ulysse Gérard, and Muluh Godson. A new era of ocaml editing: Powered by merlin, delivered via lsp. 2025.

[4] Xavier Van de Woestyne, Thomas Refis, Frédéric Bour, and Ulysse Gérard. ocaml/#14241 add merlin's typing recovery. https://github.com/ocaml/ocaml/pull/14241, 2026.

[5] Xavier Leroy and Damien Doligez and Alain Frisch and Jacques Garrigue and Didier Rémy and KC Sivaramakrishnan and Jérôme Vouillon. *The OCaml System, Release 5.5: Documentation and User's Manual*, 5.5 edition, 2026. Accessed: 2026-06-29.